OBR-39718 - Idempotency Detailed Design

- Context
- Reference Documents
- Change Control
- Open Issues
- Summary
- Release Toggle
- Production Bugs
- Requirements
  - Requirement from API contract
  - Scenarios that must be handled
  - Past defects that must be avoided (from production)
  - Issues that have prevented idempotency
  - Opportunities to create new capabilities
- Assumptions
- Out of Scope
- Key Decisions
- Platform Impacts
- Details
  - Signup Async Data Flow Diagram
  - Signup Idempotency Data Flow Diagram
  - Solution
    - Signup Consumer Use Cases
      - Initial Idempotency Check
      - Continue Processing After Initial Idempotency Check - Success
      - Continue Processing After Initial Check - Functional Failure
      - Continue Processing After Initial Check - Operational Failure DLQ Retry Exhaustion
    - Transaction History Consumer Use Cases
      - Process Signup Pending Transaction History Message
      - Process Signup Completed Transaction History Message
    - Process Subscription Consumer Use Cases
      - Process Signup Process Subscription Request Message
    - Subscription Caching Consumer Use Cases
      - Subscription Caching Process Signup Completed Transaction History Message
      - Subscription Caching Process Signup Failed Transaction History Message
    - Event Publisher Consumer Use Cases
      - Event Publisher Process Signup Completed Transaction History Message
      - Event Publisher Process Signup Failed Transaction History Message
    - Failure Event Consumer Use Cases
      - Process Failure Event Message
    - Signup Retry Consumer Use Cases
      - Process Message Retry DLQ Exhaustion
    - Subscription Event Consumer Use Cases
      - Process SIGNUP_COMPLETED_EVENT or BULK_SIGNUP_COMPLETED_EVENT
      - Process SIGNUP_FAILED_EVENT or BULK_SIGNUP_FAILED_EVENT
      - Process Any SUBSCRIPTION_RPS_EVENT
      - Process All Other Events
    - Subscription Tracking UI Uses Cases
    - Subscription Tracking API Uses Cases
    - Subscription Tracking Cron Uses Cases
  - Data Structures
    - JMS Headers
    - Documents
      - SubscriptionSignupDocument
      - SignupRequestDocument
      - SignupEventDocument
    - Subscription Signup Async DTO Library
      - SignupEventDTO
      - SignupRequestStatusEnum
      - RpsStatusEnum
      - SignupEventTypeEnum
      - EventSourceEnum
  - Test Plan
  - Work Breakdown

Context

Feature: Idempotency

Author: **Mariscal, Carlos**

Team: Gorillaz (Carlos, Bill, Kevin, Jesus, Denys)

High Level Design Document: **Idempotency High Level Design**

Epic Idempotency Phase 1: **OBR-38083 - Getting issue details...** STATUS

Design Story: **OBR-39718 - Getting issue details...** STATUS

Reference Documents

Description	Link
Mongo DB Atlas vs DocumentDB Analysis	https://www.mongodb.com/cloud/atlas/compare
DocumentDB Documentation	https://aws.amazon.com/documentdb
POC Document DB (Mongo AWS Non-SQL DB) Integration	https://github.intuit.com/WEBS/webs-pri-request-activity-model-library/commits/develop
DS-Async Use Case Scenarios	https://wiki.intuit.com/display/oBillReimagined/DS-Async+Use+Case+Scenarios
High Level Async Signup Flow	https://wiki.intuit.com/display/oBillReimagined/High+Level+Async+Signup+Flow
Temp Grants Stored Procedure Design	https://wiki.intuit.com/display/oBillReimagined/Tempgrants+Stored+Procedure
Temp Grants SQL	https://github.intuit.com/WEBS-DB/webs-dbscript/blob/master/deploy/schema/V63.1__RT4.15.0_PKG_ASYNC_TEMPGRANTSCLEANUP.sql
Signup Consumer Git Project	webs-pri-subscription-signup-async-consumer
Process Subscription Consumer Git Project	webs-pri-subscription-processsubscription-async-consumer
Subscription Caching Consumer Git Project	webs-pri-subscription-caching-consumer
Event Publisher Consumer Git Project	webs-pri-subscription-event-publisher
Failure Event Consumer Git Project	webs-pri-subscription-async-signup-failure-event-consumer
Transaction History Consumer Git Project	webs-pri-subscription-transactions-async-consumer
Signup Retry Consumer Git Project	webs-pri-signup-async-retry-consumer
Subscription Signup Async DTO Library Git Project	webs-pri-subscription-signup-async-dto-library
Detailed Design Review Recording Part 1	https://bluejeans.com/s/N9PzM

Change Control

	Date Change Requested	Details of Change Requested	Change Requestor	Story points
1	September 13, 2019	Detailed Design Review Part 1		
2	September 30, 2019	Detailed Design Review Part 2		

Open Issues

Sr. No.	Description	Date Entered	Raised By	Resolved By	Status	Resolution
1			Cheng, Edmund			

Summary

The business rationale for Idempotency solution:

1. Idempotency will resolve production bugs that occur when duplicate signup requests are published to Signup Consumer and they interfere with each other preventing a successful completion of the signup transactions.
2. Data collected to ensure Idempotency can be leveraged in the following ways:
    a. Replay poison messages after code or data fix is done to allow message to be successfully processed.
    b. Simplify Temp Grant management
    c. Track signup requests for debugging
    d. Query subscription signup's events for functional test validation.

Release Toggle

1. SIGNUP_IDEMPOTENCY_ENABLED will be used where integrating idempotency
    a. Signup Consumer upsert SubscriptionSignupDocument and doing idempotency check
    b. All components publishing subscription signup events.

Production Bugs

OBR-24916 - Getting issue details... STATUS

OBR-38718 - Getting issue details... STATUS

OBR-39133 - Getting issue details... STATUS

OBR-39682 - Getting issue details... STATUS

OBR-39704 - Getting issue details... STATUS

Requirements

Requirement from API contract

The webs_request_id header is used to provided idempotent behavior, in the event this API is called multiple times for the same customer-requested action. A scenario where this may be necessary is when the original caller's connectivity is interrupted and no reponse was received, or the caller times out, or abandons without waiting for a response. In either case the request may be resubmitted using the same value in webs_request_id as the original request and the subscription API will ensure the request is completed successfully, and only once. This value must be globally unique, such as a UUID.

Scenarios that must be handled

Request type	Required API response
Customer retry after failure event, – same request, same webs request id	Complete transaction and publish complete message to QBO
Customer retry after failure event, – same request, different webs request id	Bad Request: webs_request_id not found
Customer retry after failure event, – different request, same webs request id	If the offer is different this is an error.
Repeat request, same webs request id, after completion	Send back response message and correlation id
Repeat request, same webs request id, before completion	Send back response message after first message completes and correlation id

Concurrent requests, same time, different webs_request_id	Send back response message after first message completes and correlation id
Concurrent requests, same time, same webs_request_id	Send back response message after first message completes and correlation id
Concurrent requests, after some delay, different webs_request_id	Send back response message after first message completes and correlation id
Operational error retry from main queue, 10 second delay	Send back response message and correlation id if transaction successful
DLQ retry, 5 minute delay	Send back response message and correlation id if transaction successful
Combinations of failure retries and operational error retries	

Past defects that must be avoided (from production)

Defect	Cause
Multiple subscriptions created in WEBS DB	Two concurrent requests each create a subscription. Get Subscription returns "not found" in each case thereby allowing both transactions to create a subscription
Account already exists in BRM	The first and second requests are successful in WEBS. The second request is rejected by BRM since first request was successful processed by BRM.
Customer grants inactivated when second request fails	One scenario is that two simultaneous transaction call create account. The second to reach the Account service returns an error. This error is processed by the Failure Events Consumer and the grants just created for this customer product are inactivated.
Failure event published with incorrect correlation id	When L1 support replays transactions that failed but did not get processed by the Failure Events Consumer, the API generates a new correlation id and QBO is unable to match it to the original request.
Transactions not completed due to read time-outs, and transaction fails on DLQ retry	An internal API times out and the transaction is sent to the DLQ to retry. The internal API timed out but completed updating the database. When transaction is retried, the getAccount or getSubscription calls succeeds and a completion message is sent. However, the message never reached BRM.

Issues that have prevented idempotency

Defect	Impact
webs_request_id not available except in trans history	Currently all subsequent requests that are sent by the Offering are handled are sending back a completion message. This should only be done if the webs_request_id matches the webs_request_id of an earlier request.
With concurrent requests, the second /subsequent requests are not blocked.	This leads to the calling create account or create subscription more than once within milli-seconds of eachother. Multiple subscriptions can result, or functional failures that invalidate the grants.
Read time out on internal API's	When an internal API times out, the retry detects the subscription was create and a completion message is sent back even though the transaction did not complete the orchestration,.
Poison messages created and not processed	

Opportunities to create new capabilities

Opportunity	Description
Track all requests and report on "missing" requests	
Replay transaction at API or consumer	
Store data that supports Temp Grants, and use this to data to simplify temp grants SQL and code	

Assumptions

None

Out of Scope

None

Key Decisions

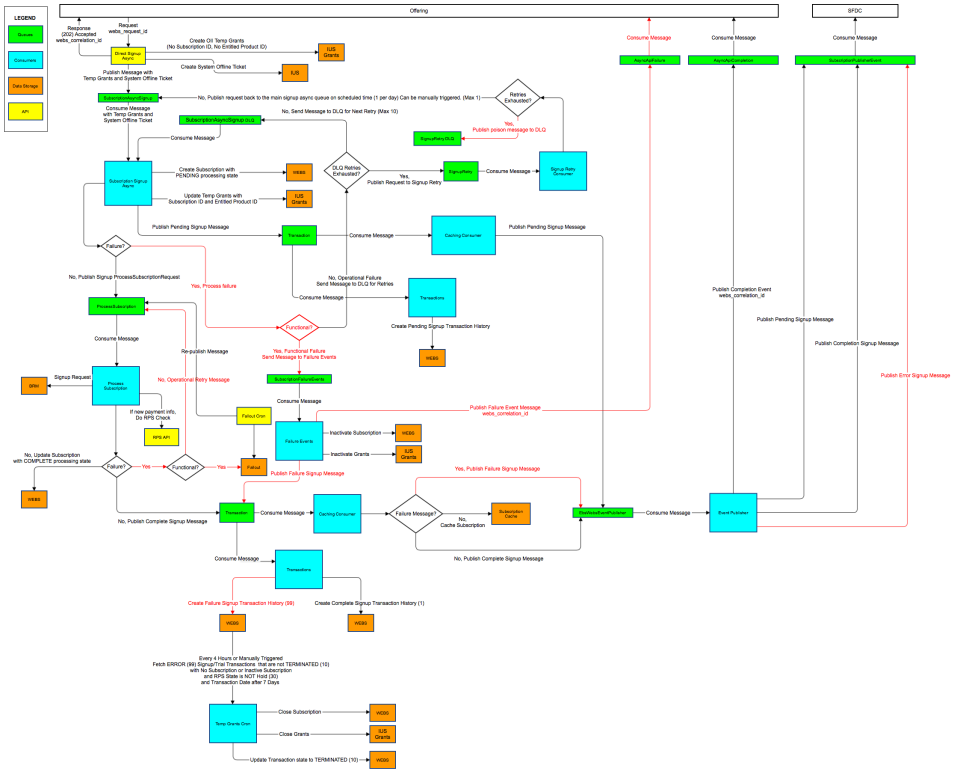
1. Document DB will be used as the SubscriptionSignupDocument data store. We currently have Document DB prototype that we can use as basis for the SDK.
2. Subscription Event Consumer will be deployed as an MSAAS artifact.
3. Subscription Tracking API will be deployed as an MSAAS artifact.
4. Subscription Tracking UI will be deployed as an App Fabric artifact.

Platform Impacts

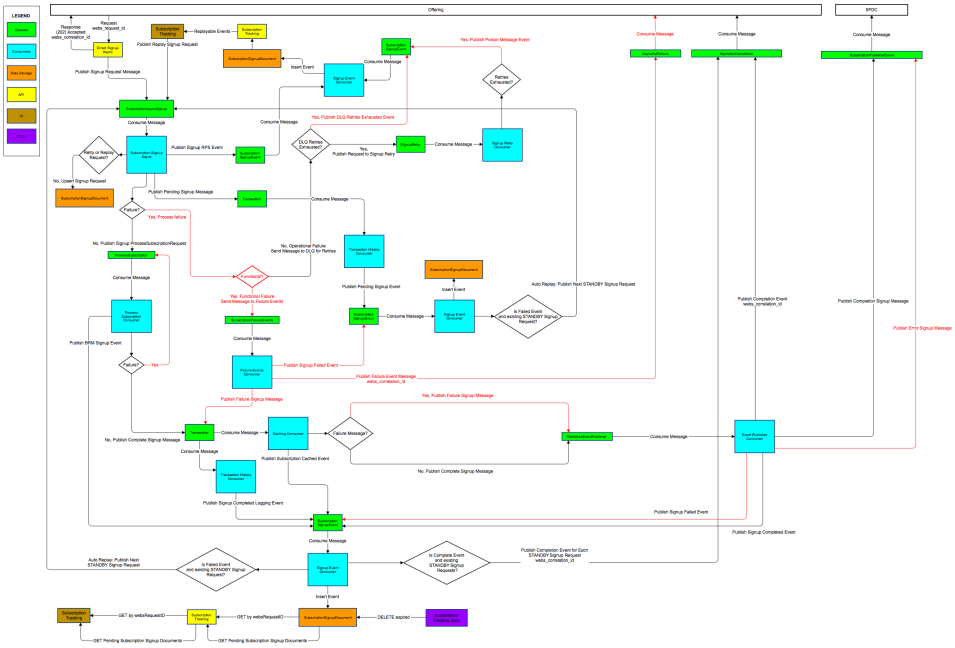
1. Create new SDK (ent-svc-cmn-document-sdk) for configuring connection to DocumentDB with low level crud operations similar to ent-svc-cmn-caching-sdk
2. Create new Library (webs-subscription-signup-document-library) to define the SubscriptionSignupDocument model and integrate with ent-svc-cmn-document-sdk to manage the document similar to webs-pri-subscription-caching-library
3. Create new Queue and DLQ (Intuit.global.prd.platform.webs.SubscriptionSignupEvent.Queue, Intuit.global.prd.platform.webs.SubscriptionSignupEvent.Queue.DLQ)
4. Create new Consumer (webs-subscription-signup-event-consumer) that consumes events from SubscriptionSignupEvent.Queue and SubscriptionSignupEvent.Queue.DLQ and integrates with webs-subscription-signup-document-library to manage SubscriptionSignupDocuments. Also contains business logic to manage Idempotent signup requests (publish completion events and publish replay events of PROCESSING signups).
    a. MSAAS artifact or new Async Service Group?
5. Create new API (webs-subscription-tracking-api) that will provide interfaces into SubscriptionSignupDocument resource.
    a. MSAAS artifact
6. Create new User Interface (webs-subscription-tracking-ui) that will provide a front end to allow PDs, L1/L2 Support and BSAs to view signup request's events and provide capability to replay repayable events.
    a. App Fabric artifact
7. Create new Cron Job that periodically deletes SubscriptionSignupDocuments.
    a. Purge completed subscriptions after 3 days.
    b. Purge failed subscriptions after 7 days.
    c. Purge all subscriptions after 14 days.
8. Update Subscription Async DTO Library to define SignupEventDTO and enumerations
    a. webs-pri-subscription-signup-async-dto-library
9. Update Signup Consumer to integrate with webs-subscription-signup-document-library to upsert new signup requests and check for idempotency.
    a. webs-pri-subscription-signup-async-consumer
10. Update existing components to publish events to the SubscriptionSignupEvent.Queue (will require Subscription Async DTO Library dependency to create events)
    a. webs-pri-subscription-signup-async-consumer
    b. webs-pri-subscription-processsubscription-async-consumer
    c. webs-pri-subscription-caching-consumer
    d. webs-pri-subscription-event-publisher
    e. webs-pri-subscription-async-signup-failure-event-consumer
    f. webs-pri-subscription-transactions-async-consumer
    g. webs-pri-signup-async-retry-consumer

Details

Signup Async Data Flow Diagram



Signup Idempotency Data Flow Diagram



Solution

Signup Consumer Use Cases

Initial Idempotency Check

1. Initial signup request message is consumed by the Signup Consumer
2. if JMS Headers **JMSXDeliveryCount, is_signup_replay** or **intuit_retry_consumer_retry_count** are NOT set or less than 1 indicating its not a retry nor a replay message then
    a. Signup Consumer upserts SubscriptionSignupDocument to Document DB adding new signupRequest (SIGNUP_EVENT) using JMS Header webs_request_id as the key
        i. websRequestID = webs_request_id (JMS Headers)
            1. signupRequest
                a. requestDateTime = current system date time (UTC) with milliseconds.
                b. websCorrelationID = webs_correlation_id (JMS Headers)
                c. requestStatus = PROCESSING
                d. requestPayload = encrypted message payload
                e. requestHeaders = encrypted message headers
                f. requestGrantOffers = message GrantOffers
                g. rpsStatus = UNKNOWN
                h. SignupEventDTO
                i. eventDateTime = current system date time (UTC) with milliseconds.
                ii. signupEventType = SIGNUP_EVENT
                iii. source = SIGNUP_CONSUMER
                iv. message = null
                v. payload = null
                vi. headers = null
    b. Document DB responds with SubscriptionSignupDocument with generated sequence number
    c. If sequence number is 1 which indicates new subscription request then
        i. Signup Consumer continues processing signup request
    d. else If sequence number is greater than 1 and each signupRequest requestStatus is FAILED except for last signupRequest (newly added during upsert) indicates offering is retrying signup request then
        i. Signup Consumer continues processing signup request
    e. else If sequence number is greater than 1 and each signupRequest requestStatus is COMPLETED except for last signupRequest then stop signup processing.
        i. Signup Consumer publishes SIGNUP_COMPLETED_EVENT to SubscriptionSignupEvent Queue
            1. SignupEventDTO
                a. eventDateTime = current system date time (UTC) with milliseconds.
                b. websRequestID = webs_request_id (JMS Headers)
                c. websCorrelationID = webs_correlation_id (JMS Headers)
                d. source = SIGNUP_CONSUMER
                e. eventType = SUBSCRIPTION_COMPLETED_EVENT
                f. message = Signup Request has already been completed.
                g. payload = Completion Event Payload
                h. headers = Completion Event Headers
        ii. Subscription Event Consumer processes SIGNUP_COMPLETED_EVENT
        iii. Stop processing signup request.
    f. else
        i. Validate to ensure that request is a valid idempotent request
            1. RealmID or AuthID must match existing PROCESSING signup request
            2. Validate idempotent request offers are matching PROCESSING signup request offers
                a. Product Codes are a subset of PROCESSING signup request offers
                b. For each matching product code, the offer's offerType are the same.
            3. If failed validation, validator throws BadRequestException to be handled by the FailureEventsConsumer
                a. Need to ensure all of the required data is collected before exception handler publishes failure event.
        ii. Update newly created signupRequest's requestStatus to STANDBY
        iii. Signup Consumer continues processing signup request
3. else
    a. if **JMSXDeliveryCount** or **intuit_retry_consumer_retry_count** exists or greater than 1 in JMS Headers
        i. Signup Consumer publishes SIGNUP_RETRY_EVENT to SubscriptionSignupEventQueue
            1. SignupEventDTO
                a. eventDateTime = current system date time (UTC) with milliseconds.
                b. websRequestID = webs_request_id (JMS Headers)
                c. websCorrelationID = webs_correlation_id (JMS Headers)
                d. source = SIGNUP_CONSUMER
                e. eventType = SIGNUP_RETRY_EVENT
                f. message = Retry Count
                g. payload = null
                h. headers = null
        ii. Subscription Event Consumer processes event.
        iii. Signup Consumer continues processing signup request retry
    b. else if **is_signup_replay** exists and is true indicating Signup Event Consumer sent automated replay signup request (Process next STA NDBY Signup) or Subscription Tracking UI sent a replay signup request
        i. Signup Consumer publishes SIGNUP_REPLAY_EVENT to SubscriptionSignupEvent Queue
            1. SignupEventDTO
                a. eventDateTime = current system date time (UTC) with milliseconds.
                b. websRequestID = webs_request_id (JMS Headers)
                c. websCorrelationID = webs_correlation_id (JMS Headers)
                d. source = SIGNUP_CONSUMER
                e. eventType = SIGNUP_REPLAY_EVENT
                f. message = null
                g. payload = null
                h. headers = null

ii. Subscription Event Consumer processes event.
iii. Signup Consumer continues processing signup request replay

Continue Processing After Initial Idempotency Check - Success

1. Signup Consumer continues to process signup request after idempotency check
2. Signup Consumer successfully validates signup request
3. Signup Consumer validates RPS status for paid signup requests
   Signup Consumer publishes SUBSCRIPTION_RPS_EVENT to SubscriptionSignupEvent Queue
   SignupEventDTO
   1. eventDateTime = current system date time (UTC) with milliseconds.
   2. websRequestID = webs_request_id (JMS Headers)
   3. websCorrelationID = webs_correlation_id (JMS Headers)
   4. source = SIGNUP_CONSUMER
      eventType = mapped RpsStatusEnum to eventType
      a. MATCH = SIGNUP_RPS_MATCH_EVENT
      b. HOLD = SIGNUP_RPS_HOLD_EVENT
      c. UNKNOWN = SIGNUP_RPS_UNKNOWN_EVENT
      d. OK = SIGNUP_RPS_OK_EVENT
   6. message = RPS message if any
   7. payload = null
   8. headers = null
   b. Subscription Event Consumer processes SUBSCRIPTION_RPS_EVENT
4. Signup Consumer successfully authorizes payment for paid signup requests
5. Signup Consumer successfully creates Subscription in WEBS database and updates Temp Grants to Full Grants in IUS
6. Signup Consumer publishes Signup Pending Message to Transactions Queue
   a. Transaction History Consumer processes Signup Pending Transaction History Message
7. Signup Consumer publishes Process Subscription Request Message to ProcessSubscription Queue
   a. Process Subscription Consumer processes Signup Process Subscription Request Message

Continue Processing After Initial Check - Functional Failure

1. Signup Consumer continues to process signup request after idempotency check
2. Functional failure is caught by Signup Consumer
   a. Signup Consumer publishes Failure Event Message to SubscriptionFailureEvents Queue
   b. Failure Event Consumer processes Failure Event Message

Continue Processing After Initial Check - Operational Failure DLQ Retry Exhaustion

1. Signup Consumer continues to process signup request after idempotency check
2. Operational failure is caught by Signup Consumer but DLQ retries are exhausted
   a. Signup Consumer publishes Signup Request Message to SignupRetry Queue
      i. Signup Retry Consumer processes Signup Request
   b. Signup Consumer publishes SIGNUP_DLQ_RETRIES_EXHAUSTED_EVENT to SubscriptionSignupEvent Queue
      i. SignupEventDTO
      1. eventDateTime = current system date time (UTC) with milliseconds.
      2. websRequestID = webs_request_id (JMS Headers)
      3. websCorrelationID = webs_correlation_id (JMS Headers)
      4. source = SIGNUP_CONSUMER
      5. eventType = SIGNUP_DLQ_RETRIES_EXHAUSTED_EVENT
      6. message = Retry Count
      7. payload = null
      8. headers = null
   c. Subscription Event Consumer processes event.

Transaction History Consumer Use Cases

Process Signup Pending Transaction History Message

1. Transaction History Consumer consumes Pending Signup Transaction Message
2. Successful insertion of Pending Signup Transaction in WEBS database
   a. Transaction History Consumer publishes SIGNUP_PENDING_LOGGING_SUCCESS_EVENT to SubscriptionSignupEvent Queue
      i. SignupEventDTO
      1. eventDateTime = current system date time (UTC) with milliseconds.
      2. websRequestID = webs_request_id (JMS Headers)
      3. websCorrelationID = webs_correlation_id (JMS Headers)
      4. source = TRANSACTION_HISTORY_CONSUMER
      5. eventType = SIGNUP_PENDING_LOGGING_SUCCESS_EVENT
      6. message = null
      7. payload = null
      8. headers = null
   b. Subscription Event Consumer processes event.
3. Failed insertion of Pending Signup Transaction in WEBS database
   a. Transaction History Consumer publishes SIGNUP_PENDING_LOGGING_FAILED_EVENT to SubscriptionSignupEvent Queue
      i. SignupEventDTO

            1. eventDateTime = current system date time (UTC) with milliseconds.
            2. websRequestID = webs_request_id (JMS Headers)
            3. websCorrelationID = webs_correlation_id (JMS Headers)
            4. source = TRANSACTION_HISTORY_CONSUMER
            5. eventType = SIGNUP_PENDING_LOGGING_FAILED_EVENT
            6. message = Failure Message
            7. payload = Transaction payload (Replayable)
            8. headers = Transaction headers (Replayable)
    b. Subscription Event Consumer processes event.

Process Signup Completed Transaction History Message

1. Transaction History Consumer consumes Completed Signup Transaction Message
2. Successful insertion of Completed Signup Transaction in WEBS database
    a. Transaction History Consumer publishes SIGNUP_COMPLETED_LOGGING_SUCCESS_EVENT to SubscriptionSignupEvent Queue
        i. SignupEventDTO
            1. eventDateTime = current system date time (UTC) with milliseconds.
            2. websRequestID = webs_request_id (JMS Headers)
            3. websCorrelationID = webs_correlation_id (JMS Headers)
            4. source = TRANSACTION_HISTORY_CONSUMER
            5. eventType = SIGNUP_COMPLETED_LOGGING_SUCCESS_EVENT
            6. message = null
            7. payload = null
            8. headers = null
    b. Subscription Event Consumer processes event.
3. Failed insertion of Completed Signup Transaction in WEBS database
    a. Transaction History Consumer publishes SIGNUP_COMPLETED_LOGGING_FAILED_EVENT to SubscriptionSignupEvent Queue
        i. SignupEventDTO
            1. eventDateTime = current system date time (UTC) with milliseconds.
            2. websRequestID = webs_request_id (JMS Headers)
            3. websCorrelationID = webs_correlation_id (JMS Headers)
            4. source = TRANSACTION_HISTORY_CONSUMER
            5. eventType = SIGNUP_COMPLETED_LOGGING_FAILED_EVENT
            6. message = Failure Message
            7. payload = Encrypted Transaction payload (Replayable)
            8. headers = Encrypted Transaction headers (Replayable)
    b. Subscription Event Consumer processes event.

Process Subscription Consumer Use Cases

Process Signup Process Subscription Request Message

1. Process Subscription Consumer consumes Signup Process Subscription Request Message
2. Successful insertion of BRM Signup Request in BRM database
    a. Process Subscription Consumer publishes BRM_SIGNUP_SUCCESS_EVENT to SubscriptionSignupEvent Queue
        SignupEventDTO
            1. eventDateTime = current system date time (UTC) with milliseconds.
            2. websRequestID = webs_request_id (JMS Headers)
            3. websCorrelationID = webs_correlation_id (JMS Headers)
            4. source = PROCESS_SUBSCRIPTION_CONSUMER
            5. eventType = BRM_SIGNUP_SUCCESS_EVENT
            6. message = null
            7. payload = null
            8. headers = null
    b. Subscription Event Consumer processes event.
    c. Process Subscription Consumer publishes Transaction History Signup Completed message to Transaction Queue
        i. Transaction History Consumer processes Signup Completed Transaction History Message
        ii. Subscription Caching Consumer processes Signup Completed Transaction History Message
3. Failed insertion of BRM Signup Request in BRM database
    a. Process Subscription Consumer publishes BRM_SIGNUP_FAILED_EVENT to SubscriptionSignupEvent Queue
        i. SignupEventDTO
            1. eventDateTime = current system date time (UTC) with milliseconds.
            2. websRequestID = webs_request_id (JMS Headers)
            3. websCorrelationID = webs_correlation_id (JMS Headers)
            4. source = PROCESS_SUBSCRIPTION_CONSUMER
            5. eventType = BRM_SIGNUP_FAILED_EVENT
            6. message = Failure Message
            7. payload = Encrypted Process Subscription Request payload (Replayable)
            8. headers = Encrypted Process Subscription Request headers (Replayable)
    b. Subscription Event Consumer processes event.
    c. Process Subscription Consumer publishes Transaction History Signup Failed message to Transaction Queue
        i. Transaction History Consumer processes Signup Failed Transaction History Message (No Idempotency Impact)
        ii. Subscription Caching Consumer processes Signup Failed Transaction History Message

---

Subscription Caching Consumer Use Cases

Subscription Caching Process Signup Completed Transaction History Message

1. Subscription Caching Consumer consumes Signup Completed Transaction History Message
2. Successful insertion of Subscription in Subscription Cache
    a. Subscription Caching Consumer publishes SUBSCRIPTION_CACHED_SUCCESS_EVENT to SubscriptionSignupEvent Queue
        SignupEventDTO
            1. eventDateTime = current system date time (UTC) with milliseconds.
            2. websRequestID = webs_request_id (JMS Headers)
            3. websCorrelationID = webs_correlation_id (JMS Headers)
            4. source = SUBSCRIPTION_CACHING_CONSUMER
            5. eventType = SUBSCRIPTION_CACHED_SUCCESS_EVENT
            6. message = null
            7. payload = null
            8. headers = null
    b. Subscription Event Consumer processes event.
    c. Subscription Caching Consumer publishes Transaction History Signup Completed message to EbsWebsEventPublisher Queue
        i. Event Publisher Process Signup Completed Transaction History Message
3. Failed insertion of Subscription in Subscription Cache
    a. Process Subscription Consumer publishes BRM_SIGNUP_FAILED_EVENT to SubscriptionSignupEvent Queue
        i. SignupEventDTO
        ii. eventDateTime = current system date time (UTC) with milliseconds.
        iii. websRequestID = webs_request_id (JMS Headers)
        iv. websCorrelationID = webs_correlation_id (JMS Headers)
        v. source = SUBSCRIPTION_CACHING_CONSUMER
        vi. eventType = SUBSCRIPTION_CACHED_FAILED_EVENT
        vii. message = Failure Message
        viii. payload = Encrypted Process Subscription Request payload (Replayable)
        ix. headers = Encrypted Process Subscription Request headers (Replayable)
    b. Subscription Event Consumer processes event.
    c. Subscription Caching Consumer publishes Transaction History Signup Completed message to EbsWebsEventPublisher Queue
        i. Event Publisher Process Signup Completed Transaction History Message

Subscription Caching Process Signup Failed Transaction History Message

1. Subscription Caching Consumer consumes Signup Failed Transaction History Message
2. Subscription Caching Consumer publishes Transaction History Signup Failed message to EbsWebsEventPublisher Queue
    a. Event Publisher Process Signup Failed Transaction History Message

---

Event Publisher Consumer Use Cases

Event Publisher Process Signup Completed Transaction History Message

1. Event Publisher Consumer consumes Signup Completed Transaction History Message
2. Successful publishing of Completion Event to AsyncApiCompletion Queue (QBO Consumes) and SubscriptionPublisherEvent Queue (SFDC Consumes)
3. Event Publisher Consumer publishes SIGNUP_COMPLETED_EVENT to SubscriptionSignupEvent Queue
    SignupEventDTO
        i. eventDateTime = current system date time (UTC) with milliseconds.
        ii. websRequestID = webs_request_id (JMS Headers)
        iii. websCorrelationID = webs_correlation_id (JMS Headers)
        iv. source = EVENT_PUBLISHER_CONSUMER
        v. eventType = SIGNUP_COMPLETED_EVENT
        vi. message = null
        vii. payload = Encrypted internal Subscriptions object payload (To be used for publishing to completion queue.)
        viii. headers = Encrypted Headers (To be used for publishing to completion queue.)
4. Subscription Event Consumer processes SIGNUP_COMPLETED_EVENT.

Event Publisher Process Signup Failed Transaction History Message

1. Subscription Caching Consumer consumes Signup Failed Transaction History Message
2. Successful publishing of Failure Event to AsyncApiFailure Queue (QBO Consumes) and SubscriptionPublisherEvent Queue (SFDC Consumes)
3. Event Publisher Consumer publishes SIGNUP_FAILED_EVENT to SubscriptionSignupEvent Queue
    SignupEventDTO
        i. eventDateTime = current system date time (UTC) with milliseconds.
        ii. websRequestID = webs_request_id (JMS Headers)
        iii. websCorrelationID = webs_correlation_id (JMS Headers)
        iv. source = EVENT_PUBLISHER_CONSUMER
        v. eventType = SIGNUP_FAILED_EVENT
        vi. message = Failure Message
        vii. payload = null
        viii. headers = null

4. Subscription Event Consumer processes SIGNUP_FAILED_EVENT.

Failure Event Consumer Use Cases

Process Failure Event Message

1. Failure Event Consumer consumes Failure Event message
2. Failure Event Consumer publishes Signup Failure message to Transaction Queue
    a. Transaction History Consumer processes Signup Failed Transaction History Message (No Idempotency Impact)
3. Failure Event Consumer publishes Failure Event message to AsyncApiFailure Queue
4. Event Publisher Consumer publishes SIGNUP_FAILED_EVENT to SubscriptionSignupEvent Queue
    SignupEventDTO
        i. eventDateTime = current system date time (UTC) with milliseconds.
        ii. websRequestID = webs_request_id (JMS Headers)
        iii. websCorrelationID = webs_correlation_id (JMS Headers)
        iv. source = FAILURE_EVENT_CONSUMER
        v. eventType = SIGNUP_FAILED_EVENT
        vi. message = Failure Message
        vii. payload = null
        viii. headers = null
5. Subscription Event Consumer processes SUBSCRIPTION_FAILED_EVENT.

Signup Retry Consumer Use Cases

Process Message Retry DLQ Exhaustion

1. Signup Retry Consumer consumes Signup Request message
2. Signup Retry Consumer verifies retry count to check if retries are exhausted
    a. Retries are exhausted
        i. Signup Retry Consumer publishes Signup Request to the SignupRetryDLQ to park the poison message (Should we still do this or just use the poison message signup event with the UI to fix message?)
        ii. Signup Retry Consumer publishes SIGNUP_POISON_MESSAGE_EVENT to SubscriptionSignupEvent Queue
            1. SignupEventDTO
                a. eventDateTime = current system date time (UTC) with milliseconds.
                b. websRequestID = webs_request_id (JMS Headers)
                c. websCorrelationID = webs_correlation_id (JMS Headers)
                d. source = SIGNUP_RETRY_CONSUMER
                e. eventType = SIGNUP_POISON_MESSAGE_EVENT
                f. message = Retry Count
                g. payload = null
                h. headers = null
        iii. Subscription Event Consumer processes event.
    b. Retries are not exhausted
        i. Signup Retry Consumer publishes Signup Request to SubscriptionAsyncSignup Queue
            1. increments intuit_retry_consumer_retry_count by 1 and adds to JMS Headers
        ii. Signup Consumer does Initial Idempotency Check

Subscription Event Consumer Use Cases

Process SIGNUP_COMPLETED_EVENT or BULK_SIGNUP_COMPLETED_EVENT

1. Subscription Event Consumer updates SubscriptionSignupDocument's signupRequest where websRequestID and websCorrelationID matches the event's values
    a. update signupRequest's requestStatus = COMPLETED
    b. insert SIGNUP_COMPLETED_EVENT to signupRequest's event list.
2. For each STANDBY signupRequest remaining
    a. Subscription Event Consumer updates signupRequest's requestStatus = COMPLETED
    b. Subscription Event Consumer inserts SIGNUP_COMPLETED_EVENT to signupRequest's event list.
    c. Subscription Event Consumer updates publishes Completion Event message to Completion Queue using payload and headers in the processed event message.

Process SIGNUP_FAILED_EVENT or BULK_SIGNUP_FAILED_EVENT

1. Subscription Event Consumer updates SubscriptionSignupDocument's signupRequest where websRequestID and websCorrelationID matches the event's values
    a. update signupRequest's requestStatus = FAILED
    b. insert SIGNUP_FAILED_EVENT to signupRequest's event list.

2. Subscription Event Consumer finds first STANDBY signupRequest in SubscriptionSignupDocument and publishes request as a replay message to SubscriptionAsyncSignup Queue
    a. adds JMS Headers **is_signup_replay** set to true value
    b. uses signupRequest's payload and headers to build replay message
3. Signup Consumer does Initial Idempotency Check

Process Any SUBSCRIPTION_RPS_EVENT

1. Update SubscriptionSignupDocument's signupRequest where websRequestID and websCorrelationID matches the event's values
    a. update signupRequest's rpsStatus = matching event types value (MATCH, HOLD, UNKNOWN or OK)
        i. SIGNUP_RPS_MATCH_EVENT = MATCH
        ii. SIGNUP_RPS_HOLD_EVENT = HOLD
        iii. SIGNUP_RPS_UNKNOWN_EVENT = UNKNOWN
        iv. SIGNUP_RPS_OK_EVENT = OK
    b. insert SUBSCRIPTION_RPS_EVENT to signupRequest's event list.

Process All Other Events

1. Insert event into SubscriptionSignupDocument's signupRequest where websRequestID and websCorrelationID matches the event's values

---

Subscription Tracking UI Uses Cases

1. App Fabric Implementation
2. Query for Subscription Signup Document by websRequestID
    a. Display list of signupRequests associated with websRequestID
        i. Display events for each signupRequest
3. Query list of Subscription Signup Documents that have PROCESSING signupRequests
    a. Provided replay ability for PROCESSING requests that have a repayable event.
        i. See SignupEventTypeEnum table's Replayable? column where value is YES for events that repayable.

Subscription Tracking API Uses Cases

1. MSAAS Implementation that integrates with Document DB
2. GET SubscriptionSignupDocument by websRequestID
3. GET list of Subscription Signup Documents that have PROCESSING signupRequests
4. GET list of Subscription Signup Documents that have STANDBY signupRequests
5. GET list of Subscription Signup Documents that have FAILED signupRequests
6. DELETE Subscription Signup Documents that are COMPLETED state and 3 days have passed since completion event
7. DELETE Subscription Signup Documents that are COMPLETED or all requests FAILED and 7 days have passed since last failed event
8. DELETE Subscription Signup Documents that are PROCESSING, STANDBY, COMPLETED or FAILED and 30 days have passed since last event.

Subscription Tracking Cron Uses Cases

1. Trigger every 24 Hours
    a. Call Subscription Tracking API to DELETE Subscription Signup Documents that are COMPLETED state and 3 days have passed since completion event
    b. Call Subscription Tracking API to DELETE Subscription Signup Documents that are COMPLETED or all requests FAILED and 7 days have passed since last failed event
    c. Call Subscription Tracking API to DELETE Subscription Signup Documents that are PROCESSING, STANDBY, COMPLETED or FAILED and 30 days have passed since last event.

Data Structures

JMS Headers

Enum	Type	Notes	Values
JMSXDeliveryCount	Integer	The request is still in flight and is being processed by one of the components in the signup request end to end flow.	Positive Number

is_signup_replay	Boolean	If true, skip idempotency check and publish otherwise do I idempotency check	true, false
intuit_retry_consumer_retry_count	Integer	Tracks how many attempts the Signup Retry Consumer has attempted to re-process the signup request	Positive Number
webs_request_id	String	Generated by Offering to uniquely identify request and check for idempotency.	UUID
webs_correlation_id	String	Uniquely identifies the request within idempotent requests. Generated by the API and returned to calling offering. Used to correlate request with completion or failure message sent by Event Publisher.	UUID

Documents

SubscriptionSignupDocument

Field	Type	Notes	Values
websRequestID	String	Generated by Offering to uniquely identify request. Used as Subscription Signup Document Key	UUID
sequence	String	Document version number generated by Document DB	Positive Number
signupRequests	SignupRequestDocument Array	List of SignupRequestDocument objects	

SignupRequestDocument

Field	Type	Notes	Values
requestDateTime	String	Timestamp stored in UTC that captures the date and time with milliseconds that the signup request was received.	
websCorrelationID	String	Uniquely identifies the request within idempotent requests. Generated by the API and returned to calling offering. Used to correlate request with completion or failure message sent by Event Publisher.	UUID
intuitTID	String	Uniquely identifies the transaction.	UUID
requestStatus	SignupRequestStatusEnum	Indicates the state of the signupRequest.	
requestPayload	String	Encrypted Payload consumed by Signup Consumer	JSON Payload
requestHeaders	String	Encrypted Headers consumed by Signup Consumer	JSON Payload
requestGrantOffers	String	Encrypted GrantOffers consumed by Signup Consumer. Array of GrantOffer = {offerID, grantID}	JSON Payload
realmID	String	user realm id to check for changing realm id	standard IUS Realm ID
authID	String	CG authID	
rpsStatus	RpsStatusEnum	Indicates the Account's RPS status	
events	SignupEventDocument Array	List of SignupEventDocument objects	

SignupEventDocument

Field	Type	Notes	Values
eventDateTime	String	Timestamp stored in UTC that captures the date and time with milliseconds that the signup event was created.	
signupEventType	SignupEventTypeEnum	Event that was processed. Used by Signup Event Consumer to determine processing required for event.	
source	EventSourceEnum	Component that reported event	
payload	String	Encrypted generated payload to store as part of event for possible replay or used to generate completion or failure event message.	JSON Payload
headers	String	Encrypted generated headers to store as part of event for possible replay or used to generate completion or failure event message.	JSON Payload

---

Subscription Signup Async DTO Library

SignupEventDTO

Field	Type	Notes	Values
eventDateTime	String	Timestamp stored in UTC that captures the date and time with milliseconds that the signup event was created.	
signupEventType	SignupEventTypeEnum	Event that was processed. Used by Signup Event Consumer to determine processing required for event.	
source	EventSourceEnum	Component that reported event	
websRequestID	String	Generated by Offering to uniquely identify request. Used as Subscription Signup Document Key. Used by Subscription Event Consumer to find SubscriptionSignupDocument.	UUID
websCorrelationID	String	Uniquely identifies the request within idempotent requests. Used by Subscription Event Consumer to find SignupRequestDocument with SubscriptionSignupDocument to add new SignupEventDocument	UUID
message	String	Any pertinent information that would be useful for tracking and debugging especially for failure type events.	
payload	String	Encrypted generated payload to store as part of event for possible replay or used to generate completion or failure event message.	JSON Payload
headers	String	Encrypted generated headers to store as part of event for possible replay or used to generate completion or failure event message.	JSON Payload

SignupRequestStatusEnum

Enum	Type	Notes
PROCESSING	String	The request is still in flight and is being processed by one of the components in the signup request end to end flow. Only one request at a time should have this state.
STANDBY	String	The request is a duplicate of another request that is currently processing and is waiting for the current processing request to complete or fail.
COMPLETED	String	The request has successfully completed and is no longer processed. Any duplicates requests with same websRequestID will also be COMPLETED. Any subsequent requests will automatically be set to COMPLETED and will not be processed.
FAILED	String	The request has failed and is no longer processed. The next duplicate STANDBY message will be automatically be replayed (Sent to the Signup Consumer with Header **is_signup_replay** set to true.)

RpsStatusEnum

Enum	Type	Notes
MATCH	String	Forbidden failure.
HOLD	String	Unauthorized failure.
UNKNOWN	String	Restricted Party Screening (RPS) check is required
OK	String	Restricted Party Screening (RPS) check is NOT required

SignupEventTypeEnum

Enum	Type	Notes	Replayable?
SIGNUP_EVENT	String	Signup Consumer adds (creates Subscription Signup Document) this event when consuming signup request message on non-retries and non-replays.	NO
SIGNUP_PENDING_LOGGING_SUCCESS_EVENT	String	Transaction History Consumer publishes this event when processing of the Signup PENDING message is successfully completed.	NO
SIGNUP_PENDING_LOGGING_FAILED_EVENT	String	Transaction History Consumer publishes this event when processing of the Signup PENDING message has failed. Possible.	YES - Send message to Transaction History Queue
BRM_SIGNUP_SUCCESS_EVENT	String	Process Subscription Consumer publishes this event after receiving successful response from BRM Signup Request.	NO
BRM_SIGNUP_FAILED_EVENT	String	Process Subscription Consumer publishes this event after receiving error response from BRM Signup Request.	YES - Send message to Process Subscription Queue
SUBSCRIPTION_CACHED_SUCCESS_EVENT	String	Subscription Caching Consumer publishes this event when processing the Signup COMPLETED message is successfully completed.	NO
SUBSCRIPTION	String	Subscription Caching Consumer publishes this event when processing the Signup COMPLETED	YES - Send message to

_CACHED_FAILED_EVENT		message has failed.	Transaction History Queue? Need to prevent reprocessing by Transaction History Consumer
SIGNUP_COMPLETED_LOGGING_SUCCESS_EVENT	String	Transaction History Consumer publishes this event when processing of the Signup COMPLETED message is done.	NO
SIGNUP_COMPLETED_LOGGING_FAILED_EVENT	String	Transaction History Consumer publishes this event when processing of the Signup COMPLETED message has failed.	YES - Send message to Transaction History Queue
SIGNUP_FAILED_EVENT	String	Failure Event Consumer publishes this event when process of the Single Signup FAILURE message is done.	NO but automatically replays next PROCESSING signupRequest if it exists in list.
SIGNUP_COMPLETED_EVENT	String	Event Publisher Consumer publishes this event when processing of the Single Signup COMPLETED message is done.	NO
BULK_SIGNUP_FAILED_EVENT	String	Failure Event Consumer publishes this event when process of the Bulk Signup FAILURE message is done.	NO but automatically replays next PROCESSING signupRequest if it exists in list.
BULK_SIGNUP_COMPLETED_EVENT	String	Event Publisher Consumer publishes this event when processing of the Bulk Signup COMPLETED message is done.	NO
SIGNUP_RETRY_EVENT	String	Signup Consumer publishes this event when processing retry signup request message. Header JMSXDeliveryCount exists in JMS Headers.	NO
SIGNUP_REPLAY_EVENT	String	Signup Consumer publishes this event when processing replay signup request message. Header is_signup_replay exists and is true.	NO
SIGNUP_DLQ_RETRIES_EXHAUSTED_EVENT	String	Signup Consumer, after DLQ retry exhaustion, publishes this event when the signup request is published to the SignupRetry Queue.	NO
SIGNUP_POISON_MESSAGE_EVENT	String	Signup Retry Consumer, after retry exhaustion, publishes this event when the signup request can no longer be processed and needs manual intervention. Payload with Headers will be sent so that User Interface can show signup request that can be replayed after changes to system or data to allow for request to be processed.	YES - Send message to SubscriptionAsyncSignup Queue
SIGNUP_RPS_MATCH_EVENT	String	Signup Consumer publishes this event when processing signup request message and RPS state is MATCH. Event will contain Forbidden failure.	NO
SIGNUP_RPS_HOLD_EVENT	String	Signup Consumer publishes this event when processing signup request message and RPS state is HOLD. Event will contain Unauthorized failure.	NO
SIGNUP_RPS_UNKNOWN_EVENT	String	Signup Consumer publishes this event when processing signup request message and RPS state is UNKNOWN. Event will message indicating that RPS Check is required.	NO
SIGNUP_RPS_OK_EVENT	String	Signup Consumer publishes this event when processing signup request message and RPS state is OK. Event will message indicating that RPS Check is NOT required.	NO

EventSourceEnum

Enum	Type	Notes
SIGNUP_CONSUMER	String	webs-pri-subscription-signup-async-consumer
PROCESS_SUBSCRIPTION_CONSUMER	String	webs-pri-subscription-processsubscription-async-consumer
SUBSCRIPTION_CACHING_CONSUMER	String	webs-pri-subscription-caching-consumer
EVENT_PUBLISHER_CONSUMER	String	webs-pri-subscription-event-publisher
FAILURE_EVENT_CONSUMER	String	webs-pri-subscription-async-signup-failure-event-consumer
TRANSACTION_HISTORY_CONSUMER	String	webs-pri-subscription-transactions-async-consumer
SIGNUP_RETRY_CONSUMER	String	webs-pri-signup-async-retry-consumer

---

Test Plan

https://wiki.intuit.com/pages/viewpage.action?spaceKey=oBillReimagined&title=Idempotency+test+scenarios+for+DS+Bulk+Signup+Async5:31 PM

Work Breakdown

User Story	Tasks	Epic	Sprint	Points	Completion Status	OBR	Driver /Crew	Notes
Create ent-svc-cmn-document-sdk	Create new SDK (ent-svc-cmn-document-sdk) for configuring connection to DocumentDB with low level crud operations similar to ent-svc-cmn-caching-sdk	OBR-41204 - Getting issue details... STATUS	2019.105 (Sep 25)	10		OBR-41567 - Getting issue details... STATUS OBR-41757 - Getting issue details... STATUS OBR-41758 - Getting issue details... STATUS OBR-41760 - Getting issue details... STATUS OBR-41759 - Getting issue details... STATUS		
Update Subscription Async DTO Library	1. Update Subscription Async DTO Library to define SignupEventDTO and enumerations a. webs-pri-subscription-signup-async-dto-library		2019.105 (Sep 25)	2		OBR-41568 - Getting issue details... STATUS		
Create webs-subscription-signup-document-library	Create new Library (webs-subscription-signup-document-library) to define the SubscriptionSignupDocument model and integrate with ent-svc-cmn-document-sdk to manage the document similar to webs-pri-subscription-caching-library		2019.106 (Oct 9)	10		OBR-41569 - Getting issue details... STATUS OBR-41762 - Getting issue details... STATUS OBR-41763 - Getting issue details... STATUS OBR-41764 - Getting issue details... STATUS OBR-41765 - Getting issue details... STATUS		
Create SubscriptionSignupEvent. Queue	Create new Queue and DLQ (Intuit.global.prd.platform.webs.SubscriptionSignupEvent.Queue, Intuit.global.prd.platform.webs.SubscriptionSignupEvent.Queue.DLQ) and add to global tomcat config properties for each environment.		2019.105 (Sep 25)	1		OBR-41570 - Getting issue details... STATUS		
Create webs-subscription-signup-event-consumer	Create new Consumer (webs-subscription-signup-event-consumer) that consumes events from SubscriptionSignupEvent.Queue and SubscriptionSignupEvent.Queue.DLQ and integrates with webs-subscription-signup-document-library to manage SubscriptionSignupDocuments. Also contains business logic to manage Idempotent signup requests (publish completion events and publish replay events of PROCESSING signups).		2019.106 (Oct 9)	10		OBR-41571 - Getting issue details... STATUS		

	a. MSAAS artifact or new Async Service Group?							
Integrate Document Library with Signup Consumer	1. Document Library Integration 2. Implement Use Cases for Single and Bulk Signup Orchestrations a. Initial Idempotency Check Use Case b. Continue Processing After Initial Idempotency Check - Success c. Continue Processing After Initial Check - Functional Failure d. Continue Processing After Initial Check - Operational Failure DLQ Retry Exhaustion		2019.106 (Oct 9)	15		OBR-41572 - Getting issue details... STATUS		
Update to Publish Events - Process Subscription			2019.107 (Oct 23)	2		OBR-41573 - Getting issue details... STATUS		
Update to Publish Events - Subscription Caching Consumer			2019.107 (Oct 23)	2		OBR-41574 - Getting issue details... STATUS		
Update to Publish Events - Event Publisher Consumer			2019.107 (Oct 23)	2		OBR-41575 - Getting issue details... STATUS		
Update to Publish Events - Failure Event Consumer			2019.107 (Oct 23)	2		OBR-41576 - Getting issue details... STATUS		
Update to Publish Events - Transaction History Consumer			2019.107 (Oct 23)	2		OBR-41577 - Getting issue details... STATUS		
Update to Publish Events - Signup Retry Consumer			2019.107 (Oct 23)	2		OBR-41578 - Getting issue details... STATUS		
Create webs-subscription-tracking-api	1. Create new API (webs-subscription-tracking-api) that will provide interfaces into SubscriptionSignupDocument resource. a. MSAAS artifact 2. Add routes for each environment		2019.107 (Oct 23)	10		OBR-41579 - Getting issue details... STATUS		
Create webs-subscription-tracking-ui	1. Create new User Interface (webs-subscription-tracking-ui) that will provide a front end to allow PDs, L1 /L2 Support and BSAs to view signup request's events and provide capability to replay repayable events. a. App Fabric artifact 2. Add routes for each environment		2019.108 (Nov 6)	30		OBR-41580 - Getting issue details... STATUS		
Create webs-subscription-tracking-cron	Create cron job that triggers deletion of SubscriptionSignupDocuments		2019.107 (Oct 23)	3		OBR-43147 - Getting issue details... STATUS		